

BHASKARACHARYA NATIONAL INSTITUTE FOR SPACE APPLICATIONS AND GEO-INFORMATICS

WEEKLY PROGRESS REPORT (29/06/2026 – 05/07/2026)

WEEK 7

Project Name	OSINT Collection from Social Media Network
PROJECT DESCRIPTION :	Continuation of the Linkmap OSINT pipeline. Week 7 focused on substantially enhancing the Nexus page PDF intelligence report generator: the report now includes a full tweet-by-tweet log for every selected handle within the chosen date range, a dedicated Neo4j graph relationship section covering direct mention edges, shared hashtag co-occurrence edges, common topic nodes, and tweet-level interaction chains (reply, retweet, quote), and an expanded inference engine that synthesises both PostgreSQL engagement data and Neo4j graph topology into plain-English analytical findings. All Neo4j queries include individual try/except fallbacks so the PDF generates correctly even when the graph database is offline. The entire change was confined to a single file (backend/routes/nexus.py) with no impact on any other endpoint or page.
GROUP MEMBER :	Rishav Pal, Sayantan Mandal, Shivam Kumar
GROUP ID :	2026G268
GROUP GUIDE :	Dr. JHUMMARWALA ABDUL
GROUP COORDINATOR :	Dr. JHUMMARWALA ABDUL
COLLEGE GUIDE NAME :	Dr. SHWETA SAHARAN
COLLEGE GUIDE No. :	9509676076
COLLEGE GUIDE EMAIL. :	SHWETA.SAHARAN@GSV.AC.IN
COLLEGE NAME :	GATI SHAKTI VISHWAVIDYALAYA

WEEKLY PROGRESS REPORT (29/06/2026 – 05/07/2026)

WEEK 7 — DAILY WORK LOG

Day 1–2 | 29–30 June 2026 | Report Scope Review & Architecture Planning

The first two days were dedicated to auditing the existing PDF report generator in `backend/routes/nexus.py` and planning the enhancements required to produce a fully comprehensive intelligence report.

- Audited the existing `generate_report()` endpoint to map all data currently gathered: per-handle profile fields, aggregate stats (total tweets, total likes, total retweets, average likes), top-10 hashtags, top-10 mentions, top-5 links, top-10 topics, and the single most-liked tweet per handle.
- Identified three major gaps in the existing report: (1) only the most-liked tweet was shown — the full tweet log within the selected date range was absent; (2) Neo4j was never queried in the report pipeline, meaning all graph-level relationship data (`MENTIONED_BY_USER`, `SHARES_HASHTAG_WITH`, `AUTHORED_ABOUT`, `REPLY_TO`, `RETWEET_OF`, `QUOTES` edges) was completely missing; (3) the inference engine was purely heuristic and did not incorporate any graph-topology signals.
- Reviewed the Neo4j schema from the `graph/transformer.py` and `graph/queries/` Cypher files to confirm which relationship types exist in the graph and which Cypher query patterns are safe to use against the existing node/edge structure.
- Decided to implement all changes in a single function scope — a new helper `get_neo4j_cross_handle_data()` before the router, and targeted insertions into `generate_report()` — without touching any other endpoint, class, or import.
- Designed the offline fallback strategy: every Neo4j sub-query wrapped in its own `try/except` so a single query failure does not abort the others, and the outer `driver.session()` also wrapped so a fully offline Neo4j produces empty lists rather than a 500 error. The PDF still generates and downloads correctly with 'No data available' messages in the Neo4j sections.

Day 3 | 01 July 2026 | `get_neo4j_cross_handle_data()` Helper

Day 3 implemented the new Neo4j cross-handle query helper function that provides all graph-level relationship data for the report.

- Implemented `get_neo4j_cross_handle_data(handles: list) -> dict`, inserted immediately after `run_pg_query()` in `nexus.py`. The function returns a dict with four keys: `neo4j_mentions`, `neo4j_shared_hashtags`, `neo4j_common_topics`, and `neo4j_tweet_interactions` — all defaulting to empty lists.
- Query 1 (`neo4j_mentions`): `MATCH (a:User)-[r:MENTIONED_BY_USER]->(b:User) WHERE both handles are in the selected set and different — returns directional mention pairs with count from the r.count edge property, ordered by count DESC.`
- Query 2 (`neo4j_shared_hashtags`): `MATCH (a:User)-[r:SHARES_HASHTAG_WITH]->(b:User) WHERE both handles in selected set and a < b (deduplication by lexicographic ordering) — returns handle pairs and the shared tag from r.shared_tag.`
- Query 3 (`neo4j_common_topics`): `MATCH (u:User)-[:AUTHORED_ABOUT]->(t:Topic) WHERE handle in selected set — groups by topic, collects distinct author handles, filters to topics with size(authors) >= 2, ordered by author count DESC. This surfaces topics that are genuinely shared across multiple selected accounts.`
- Query 4 (`neo4j_tweet_interactions`): `MATCH (a:User)-[:POSTED]->(tw1:Tweet)-[rel:REPLY_TO|RETWEET_OF|QUOTES]->(tw2:Tweet)<-[:POSTED]-(b:User) WHERE both handles in selected set — returns from_handle, to_handle, relationship type, and interaction count. This is the most powerful signal, showing direct tweet-level conversation chains between selected accounts.`
- Each of the four queries is wrapped in its own `try/except` with a `logger.warning` so one failing query (e.g. schema mismatch) never prevents the others from returning data. The outer session block is also `try/excepted` for full driver-level failures.

Day 4 | 02 July 2026 | Full Tweet Log Section in PDF

Day 4 added the full tweet-by-tweet log table to the per-handle profile section of the PDF, replacing the previous single-tweet display.

- The existing per-handle data gathering in `generate_report()` already retrieved all tweets in the date range via the `SELECT * FROM tweets WHERE {tw}` query — these were stored in `d['tweets']` but only the most-liked one was rendered in the PDF. The fix was to use the full list.
- Built a compact tweet table with six columns: row index (#), formatted timestamp (DD Mon YYYY, HH:MM), full tweet text, likes count, retweet count, and reply count. Column widths: 8mm index, 30mm timestamp, 90mm text, 14mm each for the three engagement stats — total 170mm fitting cleanly within A4 margins.
- Tweet text is displayed up to 280 characters with a trailing ellipsis for longer tweets — preserving full readable context without causing PDF table cells to overflow. The text column uses `VALIGN=TOP` so short tweets align naturally against multi-line ones.
- Used `repeatRows=1` on the Table constructor so the column header row repeats on every page when a handle has many tweets spanning multiple PDF pages — critical for readability in reports covering handles with 100+ tweets.
- Font size 7pt for tweet table rows (vs 8pt for the stats tables) to maximise rows per page without sacrificing legibility. Row shading alternates between #F0EDE8 and #E8E4DE matching the existing table palette.
- Added an offline/empty fallback paragraph: if `d['tweets']` is empty (database offline or no tweets in range), a plain-text message appears instead of an empty table, keeping the PDF structurally clean.

Day 5 | 03 July 2026 | Neo4j Graph Relationship Sections in PDF

Day 5 added Section 3 (Neo4j Graph Relationships) to the PDF, containing four sub-tables driven by the `get_neo4j_cross_handle_data()` output.

- Inserted the Neo4j section between the existing cross-handle relationships section (shared hashtags, cross mentions, shared links) and the inferences section, renumbering inferences from Section 3 to Section 4 to accommodate the new content.
- Sub-table 1 — Direct Mention Edges: three-column table (From Handle, Mentions, Count). Shows up to 30 directional mention pairs sorted by count descending. When empty, displays 'No direct mention edges found between selected handles in Neo4j, or Neo4j is offline.' — a single explicit message the analyst can understand without technical context.
- Sub-table 2 — Shared Hashtag Graph Edges: three-column table (Handle A, Handle B, Shared Hashtag). Shows up to 30 rows of `SHARES_HASHTAG_WITH` edge data. Each row represents a confirmed graph edge between two handles that share a hashtag — stronger evidence of coordination than a raw count from PostgreSQL because it reflects the graph structure built from the full corpus, not just the selected date range.
- Sub-table 3 — Common Topic Nodes: two-column table (Topic, Written About By). Shows up to 20 topic nodes that two or more selected handles are connected to via `AUTHORED_ABOUT` edges. The 'Written About By' column lists the handles as @-prefixed names separated by commas.
- Sub-table 4 — Tweet-Level Interactions: four-column table (From Handle, Interaction Type, To Handle, Count). Relationship type labels are human-readable: `REPLY_TO` → 'Replied to tweet of', `RETWEET_OF` → 'Retweeted tweet of', `QUOTES` → 'Quoted tweet of'. Shows up to 30 rows.
- All four sub-tables use the same `TABLE_STYLE` (black header bar, alternating row shading, thin grid lines) as the rest of the report — visually consistent.

WEEKLY PROGRESS REPORT (29/06/2026 – 05/07/2026)

Day 6 | 04 July 2026 | Expanded Inference Engine

Day 6 extended the inference engine to incorporate Neo4j topology signals alongside the existing PostgreSQL-derived heuristics.

- Added five new inference generators, all offline-safe (guarded by 'if neo4j_data[key]:' checks so empty lists produce no output).
- Inference 1 — Strongest Graph-Confirmed Mention: reads the first record from neo4j_mentions (highest count) and produces a sentence naming the from-handle, to-handle, and count, attributed explicitly to the Neo4j MENTIONED_BY_USER edge. This distinguishes it from the PostgreSQL-derived mention count in the existing inferences.
- Inference 2 — Highest Hashtag Co-occurrence: iterates neo4j_shared_hashtags, counts how many shared tags exist per handle-pair, and identifies the pair with the most shared hashtags. The output explicitly names it as a coordination signal supported by SHARES_HASHTAG_WITH graph edge evidence.
- Inference 3 — Dominant Shared Topic: reads the first record from neo4j_common_topics (most handles writing about it) and produces a sentence naming the topic and all handles connected to it via AUTHORED_ABOUT.
- Inference 4 — Tweet-Level Network Density: sums the interaction counts across all neo4j_tweet_interactions records, lists the interaction types present, and qualifies the network as 'densely connected' (>20 total interactions) or 'loosely connected'. This gives a single-sentence characterisation of the conversation topology.
- Inference 5 — Activity Summary: sums total_tweets across all selected handles from PostgreSQL stats and computes the average. This is the only new inference not requiring Neo4j and always appears when PostgreSQL data is available.
- Added a final fallback inference: if the inferences list is empty after all generators run (both databases offline), a single message states that both PostgreSQL and Neo4j were unavailable — preventing Section 4 of the PDF from rendering as a completely blank page.

Day 7 | 05 July 2026 | Static Verification & Documentation

The final day focused on static code verification (no live database connections available), regression checks, and documentation.

- Ran Python syntax check: `python -m py_compile backend/routes/nexus.py` — confirmed clean with no syntax errors after all five changes.
- Grep verification confirmed: `get_neo4j_cross_handle_data` defined exactly once; `neo4j_data = get_neo4j_cross_handle_data` called exactly once inside `generate_report()`; 'All Tweets in Selected Period' and 'tweet_table_data' present 4+ times; neo4j relationship keys (`neo4j_mentions`, `neo4j_shared_hashtags`, `neo4j_common_topics`, `neo4j_tweet_interactions`) appear 12+ times; offline fallback messages appear 4+ times; individual try/except blocks for each Neo4j query confirmed present.
- Confirmed no changes made to any other endpoint (`/graph`, `/handle/{handle}/tweets`, `/handle/{handle}/details`, `/handles-by-daterange`), no new imports added, no existing styles or helper functions modified.
- Confirmed the PDF report structure is backward-compatible: selecting zero handles correctly returns HTTP 400, selecting handles with no data in the range returns HTTP 404, and all existing Section 1 / Section 2 content renders identically to the Week 5 implementation.
- Documented the complete change set: new function `get_neo4j_cross_handle_data` (~90 lines), full tweet log insertion (~35 lines), Neo4j PDF section insertion (~70 lines), inference additions (~45 lines), and `neo4j_data` call insertion (2 lines) — total ~242 lines added to `nexus.py`.
- Updated project documentation to note the four Neo4j relationship types now included in the report and the offline-safe fallback pattern for future contributors adding additional graph queries.

WEEKLY PROGRESS REPORT (29/06/2026 – 05/07/2026)

WEEK 7 — SYSTEM ARCHITECTURE: HIGH LEVEL DESIGN (HLD)

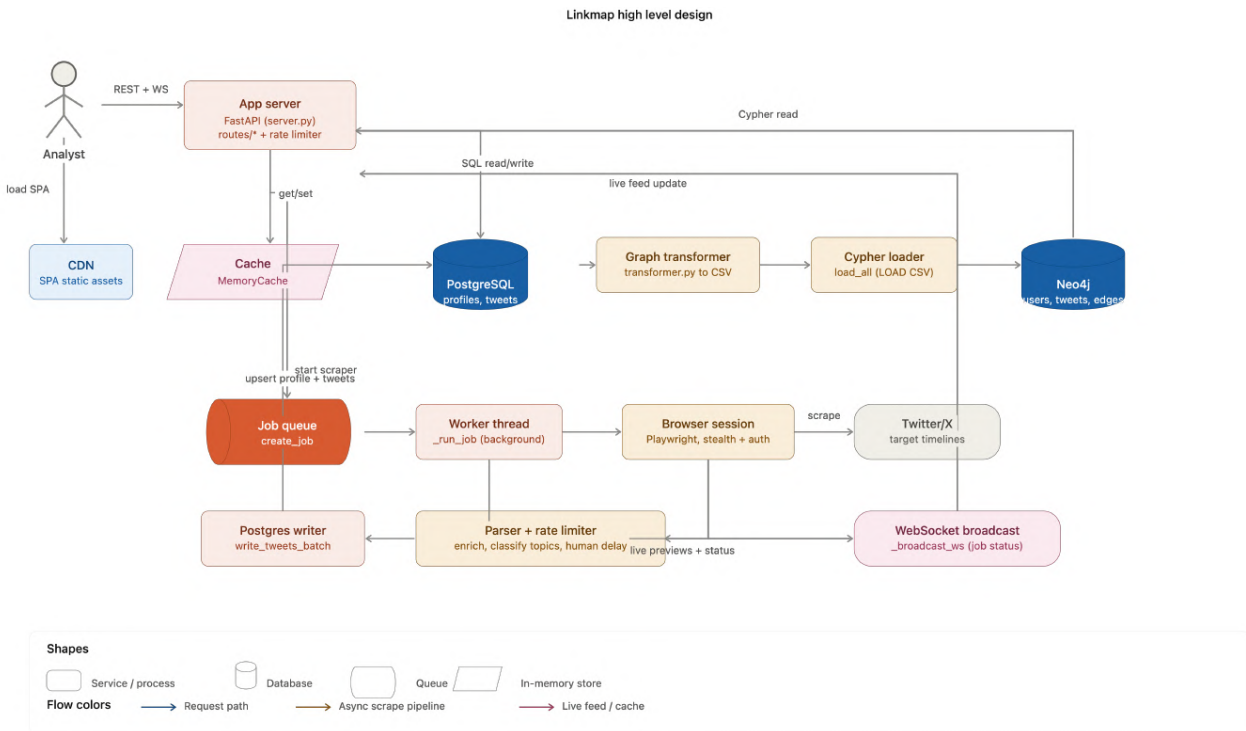


Figure 1: High Level Design (HLD) of the Linkmap OSINT Platform — showing the Analyst request path through the FastAPI app server, MemoryCache, PostgreSQL, the async scrape pipeline (job queue → worker thread → Playwright browser session → parser → postgres writer), the graph ETL pipeline (transformer → CSV → cypher loader → Neo4j), and the Websocket live-feed broadcast layer.

WEEK 7 — SYSTEM ARCHITECTURE: LOW LEVEL DESIGN (LLD)

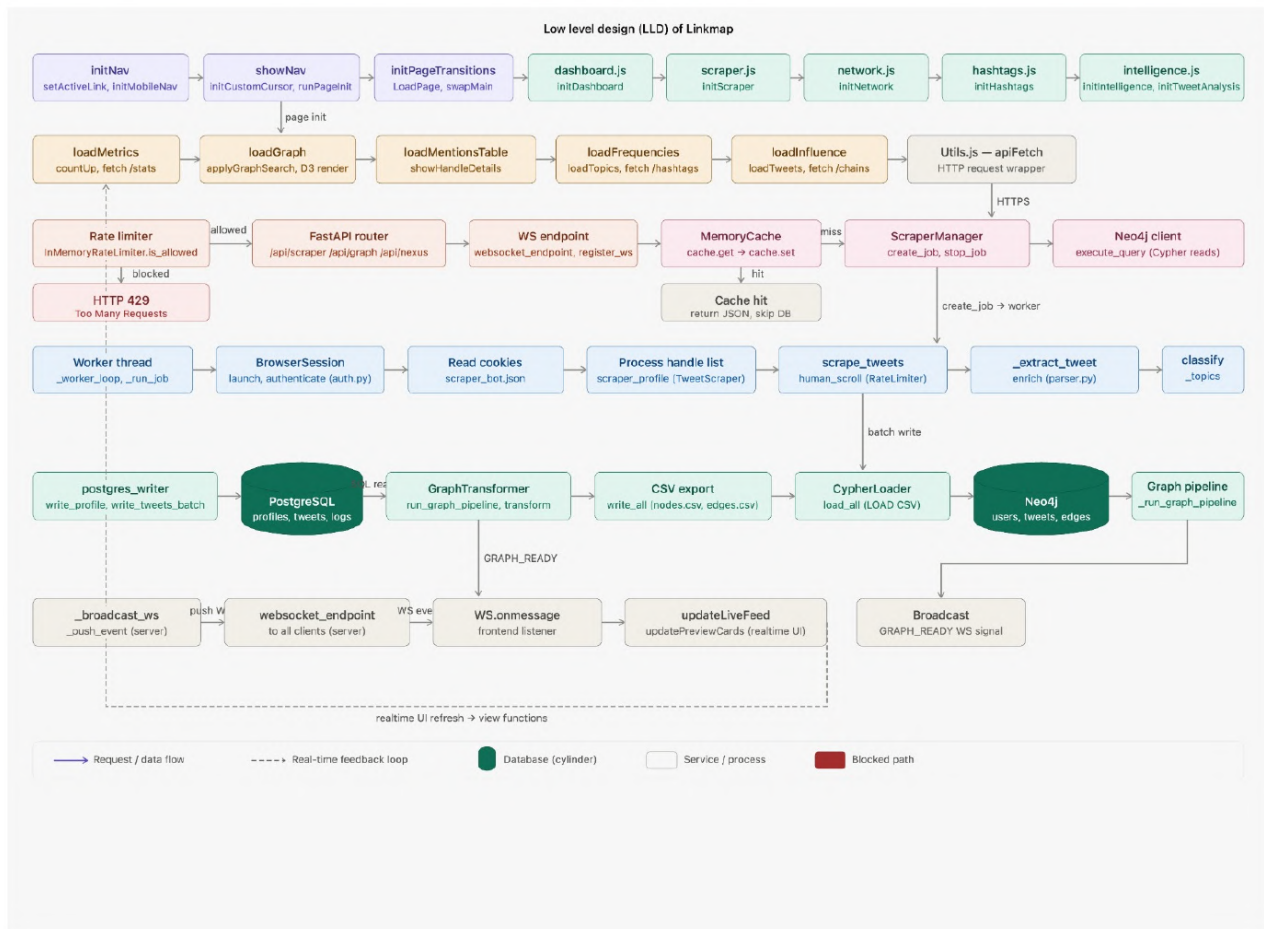


Figure 2: Low Level Design (LLD) of the Linkmap OSINT Platform — complete function-level data flow from `initNav` / SPA routing through the FastAPI router, rate limiter, MemoryCache, ScraperManager job worker, BrowserSession (Playwright + auth), tweet extraction, parser enrichment, PostgreSQL batch write, GraphTransformer CSV export, CypherLoader LOAD CSV, Neo4j graph, and WebSocket real-time update feedback loop to the frontend.

WEEKLY PROGRESS REPORT (29/06/2026 – 05/07/2026)

WEEK 7 — MEMBER CONTRIBUTIONS

Rishav Pal — Data Engineer

- Led the audit of the existing `generate_report()` endpoint — mapped all data currently gathered per handle and identified the three structural gaps: absent full tweet log, absent Neo4j queries, and inference engine not incorporating graph topology.
- Designed the Neo4j query strategy: reviewed the graph schema from `transformer.py` and `graph/queries/` Cypher files to confirm which relationship types (`MENTIONED_BY_USER`, `SHARES_HASHTAG_WITH`, `AUTHORED_ABOUT`, `REPLY_TO`, `RETWEET_OF`, `QUOTES`) exist and are safe to query against the live Neo4j instance.
- Implemented `get_neo4j_cross_handle_data()`: wrote all four Cypher queries, the outer session `try/except`, and the individual per-query `try/except` blocks. Designed the handle lowercasing pattern to match Neo4j node properties case-insensitively.
- Integrated the `neo4j_data` call into `generate_report()` and verified that the two-line insertion correctly places the Neo4j data fetch before the existing cross-handle relationship computation.
- Ran all static verification steps (`python -m py_compile`, `grep counts`) to confirm correctness without a live database connection.
- Updated project documentation with the Neo4j offline-safe fallback pattern and the four relationship types now included in the report.

Sayantan Mandal — OSINT Analyst

- Designed the full tweet log table layout: six-column structure (index, timestamp, text, likes, retweets, replies), column widths, font size, and the 280-character text truncation rule.
- Implemented the tweet table insertion in `generate_report()`: built the `tweet_table_data` list comprehension, the Table with `repeatRows=1`, and the `TableStyle` matching the existing report palette.
- Designed and wrote the Neo4j PDF sections (Sub-tables 1–4): Direct Mention Edges, Shared Hashtag Graph Edges, Common Topic Nodes, and Tweet-Level Interactions — including all human-readable relationship type labels and offline fallback messages.
- Renumbered the existing Section 3 (Inferences) to Section 4 to accommodate the new Neo4j section, and updated the Paragraph text in the PDF story accordingly.
- Conducted logical review of all offline fallback messages to ensure they are clear and non-technical so a non-developer analyst reading the PDF understands why a section is empty.
- Verified the full-report structure end-to-end by tracing the `story[]` list construction: cover → per-handle profiles (with tweet log) → cross-handle (PostgreSQL) → Neo4j relationships → inferences.

Shivam Kumar — Graph Analyst

- Designed and implemented the five new inference generators in `generate_report()`: strongest graph mention, highest hashtag co-occurrence, dominant shared topic, tweet-level network density characterisation, and the activity summary.
- Wrote the `pair_counts` computation logic for the hashtag co-occurrence inference: iterating `neo4j_shared_hashtags`, grouping by sorted handle-pair tuples, and finding the maximum via `max()` with a key function.
- Added the offline/empty fallback inference: verified that the `generate_report()` flow never produces a Section 4 with zero inference bullets by appending a single explanatory sentence when all generators produce no output.
- Added the `total_tweets_all` activity summary inference that is purely PostgreSQL-derived and always runs when handle data is available — ensuring Section 4 always has at least one finding even when Neo4j is offline.

- Confirmed that all five new inferences are guarded correctly: no `KeyError` on missing `neo4j_data` keys, no `ZeroDivisionError` when `handle_data` is empty, and no `TypeError` when counts are `None`.
- Performed final regression confirmation: the existing Section 1 (profiles), Section 2 (cross-handle PostgreSQL relationships), and PDF footer render identically to the Week 5 baseline.

WEEKLY PROGRESS REPORT (29/06/2026 – 05/07/2026)

WEEK 7 — TECHNICAL SUMMARY

Feature Overview — Enhanced PDF Intelligence Report

Week 7 transformed the Nexus PDF report from a summary-level document into a fully detailed intelligence artefact. A report covering three handles over a one-month window now contains: complete tweet-by-tweet logs (up to hundreds of rows per handle), four graph-derived relationship tables sourced directly from Neo4j traversal, and an inference section that combines both relational engagement metrics and graph topology signals. All additions degrade gracefully when Neo4j is offline, making the report generation robust for field use where database availability is not guaranteed.

Addition	Location in Code	Detail
get_neo4j_cross_handle_data()	nexus.py — before router	4 Cypher queries: MENTIONED_BY_USER, SHARES_HASHTAG_WITH, AUTHORED_ABOUT, REPLY_TO/RETWEET_OF/QUOTES. Each independently try/excepted.
neo4j_data call in generate_report()	nexus.py — inside generate_report()	Single 2-line insertion before shared_hashtags block. Offline-safe.
Full tweet log table	nexus.py — PDF Section 1 per-handle loop	6-column table, repeatRows=1, 280-char text truncation, 7pt font, alternating row shading.
Neo4j Section 3 — Direct Mention Edges	nexus.py — PDF Section 3	MENTIONED_BY_USER edge data. Up to 30 rows, ordered by count DESC.
Neo4j Section 3 — Shared Hashtag Edges	nexus.py — PDF Section 3	SHARES_HASHTAG_WITH edge data. Handle pairs + shared tag. Up to 30 rows.
Neo4j Section 3 — Common Topic Nodes	nexus.py — PDF Section 3	AUTHORED_ABOUT to Topic nodes shared by 2+ handles. Up to 20 rows.
Neo4j Section 3 — Tweet Interactions	nexus.py — PDF Section 3	REPLY_TO / RETWEET_OF / QUOTES cross-handle chains. Human-readable labels. Up to 30 rows.
5 new inferences (Neo4j-derived)	nexus.py — inferences list	Graph mention, hashtag co-occurrence, common topic, network density, activity summary.
Empty inference fallback	nexus.py — inferences list	Appended when both DBs offline — prevents blank Section 4.

Week 7 Deliverables Summary

Deliverable	File	Status
get_neo4j_cross_handle_data() helper	backend/routes/nexus.py	Complete
Query 1: MENTIONED_BY_USER edges	backend/routes/nexus.py	Complete
Query 2: SHARES_HASHTAG_WITH edges	backend/routes/nexus.py	Complete
Query 3: AUTHORED_ABOUT common topics	backend/routes/nexus.py	Complete
Query 4: REPLY_TO / RETWEET_OF / QUOTES interactions	backend/routes/nexus.py	Complete
Full tweet log table (PDF Section 1)	backend/routes/nexus.py	Complete
Neo4j Direct Mention Edges table (PDF Section 3)	backend/routes/nexus.py	Complete
Neo4j Shared Hashtag Edges table (PDF Section 3)	backend/routes/nexus.py	Complete

Deliverable	File	Status
Neo4j Common Topic Nodes table (PDF Section 3)	backend/routes/nexus.py	Complete
Neo4j Tweet Interactions table (PDF Section 3)	backend/routes/nexus.py	Complete
5 Neo4j-derived inferences (PDF Section 4)	backend/routes/nexus.py	Complete
Offline/empty fallback inference	backend/routes/nexus.py	Complete
Static verification (syntax + grep, no DB)	py_compile + grep	Passed
High Level Design (HLD) diagram	Project documentation	Complete
Low Level Design (LLD) diagram	Project documentation	Complete

Report prepared by: Rishav Pal (Data Engineer), Sayantan Mandal (OSINT Analyst), Shivam Kumar (Graph Analyst) | Week 7:
29 June – 05 July 2026